

If you choose to go down this road, you'll soon want to learn more of PostScript as a language. A good reference is [5].

CITED REFERENCES AND FURTHER READING:

Adobe Systems, Inc. 1999, *PostScript Language Reference*, 3rd ed. (Reading, MA: Addison-Wesley).[1]

Ghostscript and GSview 2007+, at <http://www.cs.wisc.edu/~ghost/>. [2]

Numerical Recipes Software 2007, "Code for PSpape and PSplot," *Numerical Recipes Webnote* No. 26, at <http://www.nr.com/webnotes?26> [3]

GNUPLOT 2007+, at <http://www.gnuplot.info>. [4]

McGilton, H., and Campione, M. 1992, *PostScript by Example* (Reading, MA: Addison-Wesley). [5]

22.2 Diagnosing Machine Parameters

A convenient fiction is that a computer's floating-point arithmetic is "accurate enough." If you believe this fiction, then numerical analysis becomes a very clean subject. Roundoff error disappears, and many finite algorithms are exact. Only manageable truncation error (§1.1) stands between you and a perfect calculation. Sounds rather naive, doesn't it?

Yes, it is naive. Notwithstanding, we have adopted this fiction throughout most of this book. To do a good job of answering the question of how roundoff error propagates, or can be bounded, for every algorithm that we have discussed would be impractical. In fact, it would not be possible: Rigorous analysis of many practical algorithms has never been made, by us or anyone.

Almost all processors today share the same floating-point data representation, namely that specified in IEEE Standard 754-1985 [1], and therefore the same strengths and weaknesses as regards roundoff error. But this was not always so! The history of computing is full of machines with strange floating-point representations by modern standards. Many early computers had 36-bit words, typically partitioned as a sign bit, 8 bits of exponent, and 27 bits of mantissa. The influential IBM 7090/7094 series was of this type. The legendary CDC 6600 and 7600 machines, designed by Seymour Cray, had 60-bit words (sign, 11-bit exponent, 48-bit mantissa). A particularly odd design was the IBM STRETCH, whose 64 bits were allocated to an exponent flag bit, 10 exponent bits, the exponent sign, a 48-bit mantissa, its sign, and three flag bits. The exponent flag bit was used to signal overflow or underflow, while the other flag bits could be set by the user to indicate — anything! So let us all be grateful for IEEE 754.

Likewise, almost all numerical computing today is done in double precision, that is, in 64-bit words, what C++ defines as `double` and we denote as `Doub`. This, also, was not always so. It has happened (one might argue) because the availability of memory has increased even more rapidly than the appetite for it in numerical computation. Many programmers born before 1960 still feel a small frisson when they type `double` instead of `float`. Indeed, the vast majority of routines in this book will work just fine, for the vast majority of applications, with merely `float` precision. In most cases, the use of `double` simply serves to reinforce an erroneous belief in the above "convenient fiction."

Still, every once in a while, you will need to know what the limitations of your machine's floating-point arithmetic actually are — the more so when your treatment of floating-point roundoff error is going to be intuitive, experimental, or casual. This will certainly be true if you ever encounter a processor with nonstandard (that is, non-IEEE compliant) hardware. Such processors still do exist, though generally hidden away in embedded special-purpose devices.

If you are lucky, then calls to the methods in the C++ standard library class `numeric_limits` will tell you what you need to know. It is a good idea to familiarize yourself with that class, including some of its esoterica, like `round_style` and `has_denorm` [2].

A more experimental approach is to use methods that were developed to ferret out machine parameters in the bad old days before standards [3,4], especially parameters that were supposed to be transparent to the (ordinary) user. The object `Machar`, listed in full in a Webnote [5], gives an implementation of a number of of these methods. The quantities determined are

- `ibeta` is the radix in which numbers are represented, almost always 2, but historically sometimes 16, or even 10.
- `it` is the number of base-`ibeta` digits in the floating-point mantissa M .
- `machep` is the exponent of the smallest (most negative) power of `ibeta` that, added to 1.0, gives something different from 1.0.
- `eps` is the floating-point number $\text{ibeta}^{\text{machep}}$, loosely referred to as the “floating-point precision.”
- `negep` is the exponent of the smallest power of `ibeta` that, subtracted from 1.0, gives something different from 1.0.
- `epsneg` is $\text{ibeta}^{\text{negep}}$, another way of defining floating-point precision. Not infrequently, `epsneg` is 0.5 times `eps`; occasionally `eps` and `epsneg` are equal.
- `ixep` is the number of bits in the exponent (including its sign or bias).
- `minexp` is the smallest (most negative) power of `ibeta` consistent with there being no leading zeros in the mantissa.
- `xmin` is the floating-point number $\text{ibeta}^{\text{minexp}}$, generally the smallest (in magnitude) useable floating value.
- `maxexp` is the smallest (positive) power of `ibeta` that causes overflow.
- `xmax` is $(1 - \text{epsneg}) \times \text{ibeta}^{\text{maxexp}}$, generally the largest (in magnitude) useable floating value.
- `irnd` returns a code in the range 0...5, giving information on what kind of rounding is done in addition, and on how underflow is handled. See below.
- `ngrd` is the number of “guard digits” used when truncating the product of two mantissas to fit the representation.

The parameter `irnd` needs some additional explanation. In the IEEE standard, bit patterns correspond to exact, “representable” numbers. The specified method for rounding an addition is to add two representable numbers “exactly,” and then round the sum to the closest representable number. If the sum is precisely halfway between two representable numbers, it should be rounded to the even one (low-order bit zero). The same behavior should hold for all the other arithmetic operations, that is, they should be done in a manner equivalent to infinite precision, and then rounded to the closest representable number.

If `irnd` returns 2 or 5, then your processor is compliant with this standard. If it

Sample Results Returned by Machar			
precision	IEEE-compliant processor		historical DEC-VAX
	float	double	
ibeta	2	2	2
it	24	53	24
machep	-23	-52	-24
eps	1.19×10^{-7}	2.22×10^{-16}	5.96×10^{-8}
negep	-24	-53	-24
epsneg	5.96×10^{-8}	1.11×10^{-16}	5.96×10^{-8}
iexp	8	11	8
minexp	-126	-1022	-128
xmin	1.18×10^{-38}	2.23×10^{-308}	2.94×10^{-39}
maxexp	128	1024	127
xmax	3.40×10^{38}	1.79×10^{308}	1.70×10^{38}
irnd	5	5	1
ngrd	0	0	0

returns 1 or 4, then it is doing some kind of rounding, but not the IEEE standard. If `irnd` returns 0 or 3, then it is truncating the result, not rounding it — not desirable.

The other issue addressed by `irnd` concerns underflow. If a floating value is less than `xmin`, many computers underflow its value to zero. Values `irnd` = 0, 1, or 2 indicate this behavior. The IEEE standard specifies a more graceful kind of underflow: As a value becomes smaller than `xmin`, its exponent is frozen at the smallest allowed value while its mantissa is decreased, acquiring leading zeros and “gracefully” losing precision. This is indicated by `irnd` = 3, 4, or 5.

Sometimes results can be compiler-dependent. For example, some compilers underflow intermediate results ungracefully, yielding `irnd` = 2 rather than 5.

Call the `report` method in `Machar` to see the comparison between its results and those returned by `numeric_limits`. Some values returned by `Machar` for IEEE compliant processors are given in the table above and compared with an important historical processor, the DEC-VAX. This processor, like its predecessor PDP-11, used a representation with a “phantom” leading 1 bit in the mantissa. You can see that this achieved a smaller `eps` for the same wordlength but could not underflow gracefully, since there were no denormalized numbers.

CITED REFERENCES AND FURTHER READING:

- IEEE Standard for Binary Floating-Point Numbers*, ANSI/IEEE Std 754–1985 (New York: IEEE, 1985).[1]
- Josuttis, N.M. 1999, *The C++ Standard Library: A Tutorial and Reference* (Boston: Addison-Wesley), §4.3.[2]